CESDE

Santiago Yosa

Ingeniero Sistemas de Información

CESDE | comfama

Unidad II

POLIMORFISMO POO

- ¿Qué es polimorfismo?

- Relación entre clases base y clases derivadas

SISTEMA CONTROL DE VERSIONES

- Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Situación - El Control Remoto Universal

Imagina que tienes un control remoto universal con un botón verde que dice "Encender". Si apuntas el control a un **Televisor** y presionas el botón, la pantalla se ilumina y suena un canal.

Si apuntas a un **Aire Acondicionado** y presionas el mismo botón, no hay pantalla, pero las aspas empiezan a girar y sale aire frío. Si apuntas a un **Equipo de Sonido**, empieza a reproducir música.

El **mensaje** (la orden "Encender") es exactamente el mismo. El control remoto no sabe cómo funciona cada aparato por dentro. Pero la **forma** en la que cada aparato responde a esa orden es única y específica.

Eso es el Polimorfismo. En programación, significa que podemos enviar el mismo mensaje (llamar al mismo método) a diferentes objetos (Hijos), y cada uno responderá a su manera, aunque los estemos controlando desde un único punto común (el Padre).

Polimorfismo

El polimorfismo (del griego "muchas formas") en POO es la capacidad que tiene un objeto de ser tratado como si fuera de un tipo más genérico (su clase base), pero al ejecutar una acción, demuestra su comportamiento específico (su clase derivada).

Por ejemplo, si `Perro` hereda de `Animal`, puedes hacer: Animal miMascota = new Perro();. Ahora, `miMascota` es de tipo `Animal`, pero en tiempo de ejecución, el programa sabe que es un `Perro`. Si llamas a `miMascota.hacerSonido()`, se ejecutará el ladrido del perro, no un sonido genérico del animal.

Para que el polimorfismo ocurra en Java, se requieren tres condiciones no negociables:

1. **Herencia:** Debe existir una jerarquía de clases (Padre e Hija).
2. **Sobreescritura (@Override):** La clase hija debe redefinir un comportamiento del padre.
3. **Referencia de Clase Base (Upcasting):** Aquí está la clave técnica. Declaramos la variable del tipo *Padre*, pero instanciamos el objeto del tipo *Hijo*.
   ClasePadre variable = new ClaseHija();

Upcasting y la Gestión de Memoria (Stack vs. Heap)

Animal miMascota = new Perro();

- **Tiempo de Compilación (Static Check):** El compilador mira la variable miMascota. Ve que es de tipo Animal. Verifica si la clase Animal tiene el método comer(). Si lo tiene, compila. El compilador *no sabe* que en realidad es un perro.

- **Tiempo de Ejecución (Dynamic/Late Binding):** La JVM mira en el **Heap** (memoria dinámica). Encuentra que el objeto real creado con new es un Perro. Ignora el método de Animal y ejecuta el método comer() de Perro.

Regla de oro: El tipo de la referencia (lado izquierdo) determina *qué* métodos puedes invocar. El tipo del objeto (lado derecho) determina *cuál* implementación se ejecuta.

Anatomía del Código

```
// CLASE BASE
class MetodoPago {
   void procesar(double monto) {
      System.out.println("Pago genérico de: " + monto);
   }
}

// CLASE DERIVADA 1
class TarjetaCredito extends MetodoPago {
   @Override
   void procesar(double monto) {
      System.out.println("Cobrando $" + monto + " a la tarjeta VISA terminada en 1234");
   }
}

// CLASE DERIVADA 2
class PayPal extends MetodoPago {
   @Override
   void procesar(double monto) {
      System.out.println("Redirigiendo a PayPal para cobrar $" + monto);
   }
}

// EJECUCIÓN (Polimorfismo)
public class Main {
   public static void main(String[] args) {
      // Upcasting: La referencia es MetodoPago, el objeto es TarjetaCredito
      MetodoPago pago1 = new TarjetaCredito();

      // Upcasting: La referencia es MetodoPago, el objeto es PayPal
      MetodoPago pago2 = new PayPal();

      // LATE BINDING: En ejecución, Java busca la versión del objeto real
      pago1.procesar(100); // Salida: Cobrando $100 a la tarjeta...
      pago2.procesar(200); // Salida: Redirigiendo a PayPal...
   }
```

La Relación Indispensable con la Herencia

El polimorfismo **depende** de la herencia (y/o interfaces). La herencia crea la relación **"ES UN/A"** que es la base. Un Coche **ES UN** Vehiculo. Esto es lo que permite que una variable de tipo Vehiculo pueda legalmente almacenar una referencia a un objeto Coche. Sin esta relación, el polimorfismo no puede existir.

Tipos de Polimorfismo en Java

Polimorfismo Estático (en tiempo de compilación): También conocido como **sobrecarga de métodos** (*method overloading*).

Polimorfismo Dinámico (en tiempo de ejecución): El verdadero protagonista. También conocido como **sobrescritura de métodos** (*method overriding*).

Polimorfismo Estático (en tiempo de compilación), sobrecarga de métodos (Method Overloading)

- Ocurre cuando hay múltiples métodos **en la misma clase** con el mismo nombre pero con **diferentes parámetros** (ya sea en cantidad o tipo).
- Java decide cuál método llamar en el momento de la **compilación**, basándose en los argumentos que le pasas.
- Ejemplo: System.out.println() está sobrecargado para aceptar strings, ints, doubles, etc.

Polimorfismo Dinámico (en tiempo de ejecución):

- Ocurre cuando una subclase provee una implementación específica de un método que ya está definido en su superclase.
- Java decide cuál método llamar en el **momento de la ejecución**, basándose en el tipo real del objeto en memoria. **Este es el que permite el código extensible y mantenible**.

Sobrecargar (Overload): Ocurre en la **misma clase**. Tienes varios métodos con el **mismo nombre** pero con **diferentes parámetros** (distinta cantidad o tipo). Sirve para ofrecer la misma operación con diferentes tipos de datos.

Sobrescribir (Override): Ocurre en una **clase hija (subclase)**. Reemplazas un método de la clase padre con una nueva implementación. El método debe tener el **mismo nombre y los mismos parámetros** que el del padre. Es la base del polimorfismo.

Ejemplo Práctico

```
// 1. EL PADRE (El tipo de Referencia)
public class Empleado {
   public void trabajar() {
      System.out.println("El empleado trabaja genéricamente.");
   }
}

// 2. LOS HIJOS (Los tipos de Objeto real)
public class Disenador extends Empleado {
   @Override
   public void trabajar() {
      System.out.println("El diseñador crea interfaces en Figma.");
   }
}

public class Programador extends Empleado {
   @Override
   public void trabajar() {
      System.out.println("El programador escribe código en Java.");
   }
}


import java.util.ArrayList;
import java.util.List;

public class SistemaEmpresa {
   public static void main(String[] args) {
      // 1. La Lista Genérica (El poder del Polimorfismo)
      // Acepta a cualquier objeto que "SEA UN" Empleado
      List<Empleado> nomina = new ArrayList<>();

      // 2. Upcasting automático al agregar a la lista
      nomina.add(new Empleado());
      nomina.add(new Disenador());
      nomina.add(new Programador());

      // 3. Ejecución Polimórfica
      System.out.println("--- INICIANDO JORNADA LABORAL ---");
      for (Empleado trabajador : nomina) {
         // Un solo mensaje, múltiples comportamientos
         trabajador.trabajar();
      }
   }
}
```

¿Cuál es la diferencia real entre sobrescribir (overriding) y sobrecargar (overloading)?

La **sobrescritura** (polimorfismo dinámico) ocurre en clases diferentes (base y derivada), tiene la misma firma de método y se resuelve en tiempo de ejecución. Su propósito es cambiar o especializar el comportamiento de un método heredado.

La **sobrecarga** (polimorfismo estático) ocurre en la misma clase, tiene diferentes firmas de método (distintos parámetros) y se resuelve en tiempo de compilación. Su propósito es ofrecer diferentes maneras de llamar a un método con el mismo nombre.

Preguntas Frecuentes

¿Por qué complicarme haciendo MetodoPago p = new PayPal() en lugar de PayPal p = new PayPal()?

Por **desacoplamiento**. Si tu sistema está diseñado para recibir MetodoPago, puedes crear una lista List<MetodoPago>. Esto te permite recorrer la lista y ejecutar procesar() sin importar si son tarjetas, efectivo o PayPal. Si usas el tipo específico, tu código se vuelve rígido y tendrías que hacer un if por cada tipo de pago.

Si uso Animal a = new Perro(), ¿puedo llamar a un método exclusivo de Perro como ladrar()?

No directamente. Como la referencia es Animal, el compilador solo te deja usar lo que existe en Animal. Si necesitas acceder a ladrar(), tendrías que hacer un *Downcasting* explícito (((Perro)a).ladrar()), pero esto se considera una mala práctica si se abusa, ya que rompe la abstracción del polimorfismo.

Práctica acompañada

Sistema de Figuras Geométricas para un editor gráfico. Vamos a crear un sistema que calcule áreas, pero no sabemos qué figuras dibujará el usuario.

Instrucciones

1. Crea un proyecto Maven/Java nuevo.
2. Crea una clase padre llamada Figura.
   - Atributo: String color.
   - Método: double calcularArea() que retorne 0.0 (o imprima "Área genérica").
3. Crea una clase hija Circulo que hereda de Figura.
   - Atributo: double radio.
   - Constructor: recibe color y radio.
   - Sobreescribe (@Override) calcularArea(): retorna Math.PI * radio * radio.
4. Crea una clase hija Rectangulo que hereda de Figura.
   - Atributos: base, altura.
   - Sobreescribe calcularArea(): retorna base * altura.
5. En el Main:
   - Crea una lista: List<Figura> misFiguras = new ArrayList<>();.
   - Agrega un círculo y un rectángulo a la lista.
   - Recorre la lista e imprime el área de cada uno.

```java
import java.util.ArrayList;
import java.util.List;
// 1. Clase Base
class Figura {
    protected String color;
    public Figura(String color) {
        this.color = color;
    }
    public double calcularArea() {
        return 0.0;
    }
}
// 2. Clases Derivadas
class Circulo extends Figura {
    private double radio;
    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }
    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
class Rectangulo extends Figura {
    private double base;
    private double altura;
    public Rectangulo(String color, double base, double altura) {
        super(color);
        this.base = base;
        this.altura = altura;
    }
    @Override
    public double calcularArea() {
        return base * altura;
    }
}
// 3. Ejecución
public class EditorGrafico {
    public static void main(String[] args) {
        List<Figura> lienzo = new ArrayList<>();
        // Polimorfismo: Guardamos hijos en una lista del tipo Padre
        lienzo.add(new Circulo("Rojo", 5.0));
        lienzo.add(new Rectangulo("Azul", 4.0, 6.0));
        // Bucle polimórfico
        for (Figura f : lienzo) {
            System.out.println("Figura de color " + f.color);
            // Late Binding: Decide qué método usar en tiempo de ejecución
            System.out.println("Área: " + f.calcularArea());
            System.out.println("-----------------");
        }
    }
}
```

Práctica Learning by Doing - Ejemplo de Aprendizaje (Gestor de Archivos local)

Instrucciones para el estudiante:

1. Crea una clase base Archivo con un método abrir().
2. Crea dos clases hijas: ArchivoPDF y ArchivoImagen. Ambas deben sobrescribir el método abrir().
3. En tu main, crea un arreglo o ArrayList de tipo Archivo. Añade un PDF y una Imagen. Recórrelo y llama a abrir().

```
class Archivo {
   public void abrir() { System.out.println("Abriendo archivo genérico..."); }
}
class ArchivoPDF extends Archivo {
   @Override
   public void abrir() { System.out.println("Abriendo documento en Adobe Reader..."); }
}
class ArchivoImagen extends Archivo {
   @Override
   public void abrir() { System.out.println("Mostrando imagen en la Galería..."); }
}
public class SistemaOS {
   public static void main(String[] args) {
      Archivo[] carpeta = {new ArchivoPDF(), new ArchivoImagen()};
      for(Archivo a : carpeta) {
         a.abrir(); // Polimorfismo en acción
      }
   }
}
```

Práctica B: Afianzamiento (Trabajo Colaborativo en GitHub)

En un repositorio público llamado zoologico-polimorfico. Crearemos una clase base Animal (con método hacerSonido()) y una clase Main con un ArrayList<Animal>. Se debe hacer un commit inicial y subir.

Instrucciones:

1. **Clonar:** Copia el repositorio del profesor en tu máquina. git clone <URL_DEL_PROFE>
2. **Aislar el trabajo (Branch):** NUNCA trabajes en la rama main de un proyecto colaborativo. Crea tu propia rama. git checkout -b feat-leon-tuNombre
3. **Codificar:** Crea la clase Leon extends Animal y sobrescribe el método hacerSonido() para que imprima el sonido del león. En el Main, añade tu León a la lista genérica.
4. **Versionar:** git add . git commit -m "feat: agregar clase Leon y comportamiento polimorfico"
5. **Subir la rama:** git push origin feat-leon-tuNombre
6. **Pull Request (PR):** Ve a GitHub, abre un "Pull Request" pidiendo al profesor que fusione tu rama con la principal.
7. **(El docente aprueba el PR de un estudiante en pantalla).**
8. **Actualizar el local (fetch y pull):** Ahora que el main remoto tiene el código de tu compañero, tu máquina está desactualizada.
    - git checkout main (Vuelves a tu rama principal).
    - git fetch (Pregunta al servidor qué cambios nuevos hay, pero no los aplica aún).
    - git pull (Descarga y fusiona los cambios de tu compañero en tu máquina).
    - Ejecuta el Main y verás el León de tu compañero funcionando gracias al polimorfismo

Repositorio Semilla

Clase padre

```
public class Animal {
    // Protected para que los hijos puedan acceder al nombre si lo necesitan
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    // Método que será sobrescrito (Polimorfismo)
    public void hacerSonido() {
        System.out.println(nombre + " hace un sonido genérico.");
    }

    public String getNombre() {
        return nombre;
    }
}
```

Repositorio Semilla

Clase hijo

```
public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre);
    }
    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Guau, guau!");
    }
}
```

Clase main MainZoologico

```
import java.util.ArrayList;
import java.util.List;

public class MainZoologico {
   public static void main(String[] args) {

      // 1. La lista polimórfica (Acepta cualquier hijo de Animal)
      List<Animal> zoologico = new ArrayList<>();

      // 2. Poblado inicial del zoológico
      zoologico.add(new Perro("Firulais"));

      // ========================================================
      // ZONA DE TRABAJO ESTUDIANTIL
      // Instrucción: Cada estudiante debe instanciar su animal aquí
      // Ejemplo: zoologico.add(new Leon("Simba"));
      // ========================================================


      // 3. Ejecución del Polimorfismo
      System.out.println("--- INICIANDO RECORRIDO POR EL ZOOLÓGICO ---");
      for (Animal animal : zoologico) {
         animal.hacerSonido(); // Enlace Dinámico en acción
      }
   }
}
```

En su rama local

git clone, luego git checkout -b feat-leon-tuNombre

Crear su propia clase

Modificar el MainZoologico.java

Prevención de Conflictos Severos: Crearemos un archivo NUEVO (Leon.java, Gato.java, Vaca.java), así Git no generará conflictos en esos archivos porque son únicos de cada rama.

El Conflicto Controlado: Donde **SÍ** habrá un conflicto de fusión es en MainZoologico.java, porque varios estudiantes van a agregar su zoologico.add(...) en la misma línea.

El Cierre (El Fetch/Pull): Luego de resolver los conflictos en GitHub, hagan git checkout main y luego git pull.

¿Qué pasa al ejecu al ejecutar su MainZoologico local?

Troubleshooting (Solución de Errores Comunes)

Error Conceptual: "No entiendo por qué git fetch no actualiza mis archivos".

- *Solución:* Explicar que fetch es como mirar por la ventana para ver si llegó el correo (actualiza el historial de Git). pull es salir, tomar el correo y abrirlo sobre tu escritorio (fusiona los archivos).

Conflicto de Fusión (Merge Conflict): Varios estudiantes modificaron la misma línea del ArrayList en el Main.

- *Solución:* El IDE mostrará código con <<<<<<< HEAD. Enseñarles a no entrar en pánico, leer las dos versiones, decidir cuál se queda (o fusionar ambas) y hacer un nuevo commit de resolución.

Fallo de Polimorfismo: Intentar llamar a un método específico del hijo desde la lista.

- *Error:* animal.cazar() (Si cazar solo está en Leon).
- *Solución:* Recordar el contrato del Upcasting. El compilador verifica contra el molde del Padre.

¿Debería saber algo más?

1. **Clases Abstractas:**
   El ejemplo de Figura, tener un método calcularArea() que retorne 0.0 en la clase padre es un poco "sucio" y peligroso (¿y si alguien instancia una Figura genérica?). La solución profesional es hacer la clase abstract y el método abstract, obligando a los hijos a implementar la lógica y prohibiendo instanciar al padre.

2. **Principio de Sustitución de Liskov (LSP):**
   Es la "L" de SOLID. Explica brevemente que el polimorfismo no solo es compilación, es semántica. Si B hereda de A, B debe poder usarse en cualquier lugar donde se use A sin romper la aplicación. Si tu sobreescritura cambia radicalmente lo que se espera del método (ej: en vez de calcular área, borra el disco duro), violas el polimorfismo lógico.

CESDE | comfama